



Using HTTP Adapters in Productions

Version 2026.1
2026-04-20

Using HTTP Adapters in Productions
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1
Copyright © 2026 InterSystems Corporation
All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

1 About the HTTP Adapters	1
1.1 HTTP Inbound Adapter and Helper Classes	1
1.2 HTTP Outbound Adapter and Helper Classes	2
1.3 See Also	3
2 Using the HTTP Inbound Adapter	5
2.1 Overall Behavior	5
2.2 Creating a Business Service to Use the HTTP Inbound Adapter	7
2.3 Implementing the OnProcessInput() Method	8
2.4 Implementing the OnErrorStream() Method	9
2.5 Using the HTTP Request	9
2.5.1 About the Attributes Array	10
2.5.2 Retrieving Form Variables from a POST Request	11
2.6 Adding and Configuring the Business Service	11
2.6.1 Specifying the Sources of HTTP Requests	12
2.6.2 Specifying the Character Set to Use	12
2.7 See Also	12
3 Using the HTTP Outbound Adapter	13
3.1 Overall Behavior	13
3.2 Creating a Business Operation to Use the Adapter	13
3.3 Creating Message Handlers	15
3.4 Calling HTTP Commands	16
3.4.1 Sending Form Data	16
3.4.2 Sending a Request Body	16
3.4.3 Reference Information for HTTP Methods	17
3.4.4 Handling the HTTP Response	20
3.5 Managing Cookies	20
3.6 Creating Custom HTTP Requests	21
3.7 Using the HTTP Response	21
3.8 Examples	22
3.8.1 Example with Post	22
3.8.2 Example with Get	23
3.9 Adding and Configuring the Business Operation	23
3.9.1 Specifying the Destination Server and URL Path	24
3.9.2 Specifying a Proxy Server	24
3.10 See Also	24
4 Built-in HTTP Components	25
4.1 Available HTTP Business Hosts	25
4.2 Generic HTTP Message Class	25
4.3 Target URL of the HTTP Request	26
4.4 URL Parameters	26
4.5 More About Message Headers	26
4.6 See Also	26
HTTP Adapter Settings	29
Settings for the HTTP Inbound Adapter	30
Settings for the HTTP Outbound Adapter	33

1

About the HTTP Adapters

The HTTP adapters (EnsLib.HTTP.InboundAdapter and EnsLib.HTTP.OutboundAdapter) enable your production to send and receive HTTP requests and responses. This page provides a brief introduction to these adapters.

Note: Although InterSystems strongly recommends that you use the Web Gateway to set up communication between a web server and InterSystems IRIS® data platform, you can use the HTTP inbound adapter if needed. Note that this is not a replacement for a web server; it does not, for example, support proxy forwarding.

Tip: InterSystems IRIS also provides specialized business service classes that use these adapters, and one of those might be suitable for your needs. If so, no programming would be needed. See [Connectivity Options](#).

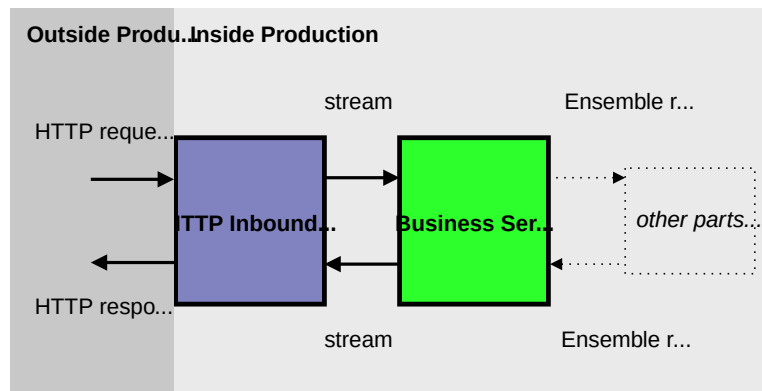
1.1 HTTP Inbound Adapter and Helper Classes

The EnsLib.HTTP.InboundAdapter is the HTTP listener for custom port listening, XML listening, and/or raw HTML handling. The inbound adapter listens on a private port rather than using a CSP page (which uses the standard web server to handle HTTP requests). For more information about the Web Gateway, see [Installing the Web Gateway](#).

The EnsLib.HTTP.InboundAdapter class provides runtime settings that you use to specify items like the following:

- A local port, where the adapter will listen for input
- A list of IP addresses from which the adapter will accept input (if you want to restrict the possible sources)
- Settings that specify whether to use the character set given in the inbound request, and if not, what other character set to use

The inbound HTTP adapter listens on the specified port, reads the input, and sends the input as a stream (either binary or character, depending on the character set in use) to the associated business service. The business service, which you create and configure, uses this stream and communicates with the rest of the production.



When you work with the HTTP inbound adapter, there are two helper classes that you might use: `%Library.GlobalCharacterStream` and `%Library.GlobalBinaryStream`. The inbound adapter sends a stream to the associated business service. Specifically, this is an instance of `%Library.GlobalCharacterStream` or `%Library.GlobalBinaryStream`, depending on the character set being used. In general, these classes provide methods that you can use to read the contents of the stream, get the length of the stream, read a single line, rewind, append data, and so on. For information on these classes, see [Working with Streams](#).

1.2 HTTP Outbound Adapter and Helper Classes

`EnsLib.HTTP.OutboundAdapter` is the adapter for sending HTTP requests outside a production and receiving HTTP responses. This adapter provides settings to control items such as the following:

- Server and port to which the adapter will send HTTP requests
- URL path for the resource to request, at the given server and port
- An optional TLS configuration to use for the connection to the server
- Optional information to specify a proxy server through which the adapter can route requests

The adapter provides methods for sending HTTP POST, GET, and PUT actions:

- The main methods are `PostFormData()` and `GetFormData()`. Each accepts an output argument for the response object, a comma-separated list of form variable names, and a variable number of form variable arguments, one for each of the names in the comma-separated list. If you want to set multiple values for a form variable, you can use the same name multiple times in the list. Of course, you can also use these methods with no form variables to request flat scalar content such as a regular web page.
- For situations with a complicated set of form variables, use the methods `PostFormDataArray()` and `GetFormDataArray()`. These methods accept a multidimensional array instead of a variable argument list. This can help keep things organized, because you can provide multiple values for a given form variable as subnodes in the array, rather than as multiple entries in the list of names. You can also index the array by form variable name rather than by position.
- The low-level worker method `SendFormDataArray()` is available for situations when you need to use a PUT or some other unusual HTTP request, or where you need to customize aspects of your HTTP request other than form variables or cookies.

The adapter also provides properties and methods to manage cookies.

When you work with the HTTP outbound adapter, there are two helper classes that you can use:

- The HTTP outbound adapter uses `%Net.HttpRequest` to encapsulate the HTTP request that it sends.

If you use `PostFormData()`, `GetFormData()`, `PostFormDataArray()`, or `GetFormDataArray()`, the adapter creates the request automatically and you do not have direct access to it.

However, if you use `SendFormDataArray()`, you can create an instance of the `%Net.HttpRequest` class, set its properties, and use it to initialize the HTTP request that you send in that method. You use this technique when you need to set properties of the HTTP request (such as the proxy authentication) that cannot be set via the adapter.

- In all cases, the HTTP response is encapsulated in an instance of `%Net.HttpResponse`. The `%Net.HttpResponse` class provides methods to access the HTTP headers, as well as properties that contain the body of the response (a stream object), reason codes, the HTTP version of the server, and so on.

1.3 See Also

- [Using the HTTP Inbound Adapter](#)
- [Using the HTTP Outbound Adapter](#)
- [Business Host Classes That Use HTTP and SOAP Adapters](#)
- [Built-in HTTP Components](#)

2

Using the HTTP Inbound Adapter

This page describes the default behavior of the HTTP inbound adapter (`EnsLib.HTTP.InboundAdapter`) and describes how to use this adapter in your productions.

Tip: InterSystems IRIS® data platform also provides specialized business service classes that use this adapter, and one of those might be suitable for your needs. If so, no programming would be needed. See [Built-in HTTP Components](#) and [Business Host Classes That Use HTTP and SOAP Adapters](#).

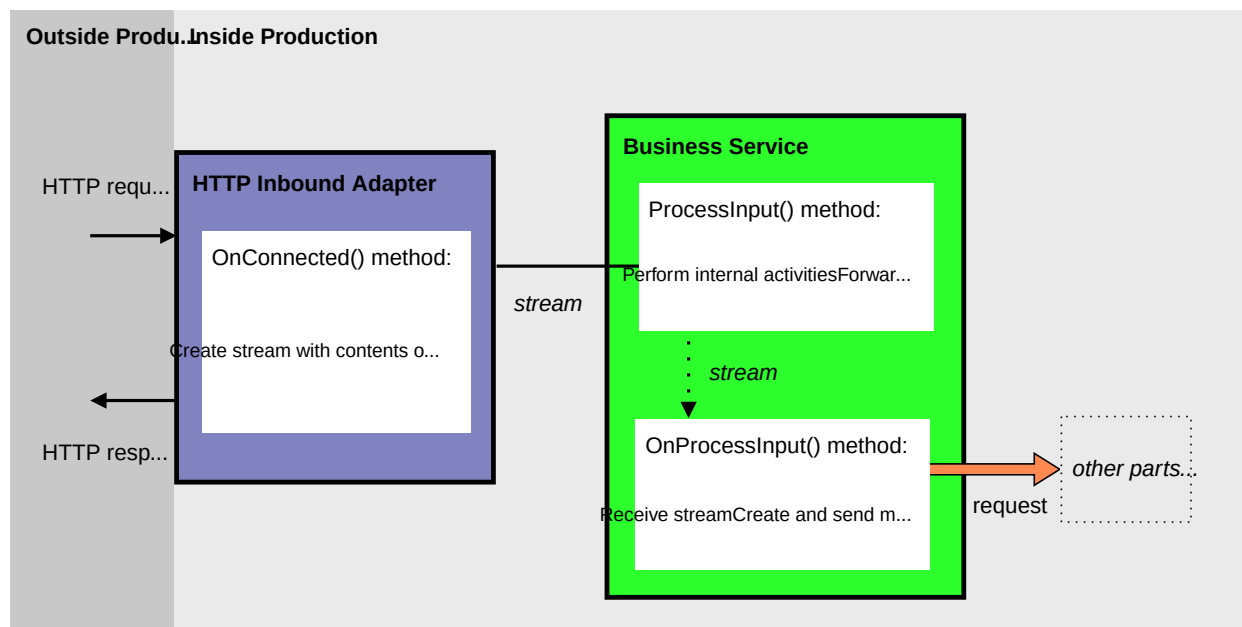
2.1 Overall Behavior

`EnsLib.HTTP.InboundAdapter` is an HTTP listener for custom port listening, XML listening, and/or raw HTML handling. You use this adapter in cases when you prefer to listen on a private port rather than using a CSP page (which uses your configured web server to handle HTTP requests).

First, the class provides runtime settings that you use to specify items like the following:

- A local port, where the adapter will listen for input
- A list of IP addresses from which the adapter will accept input (if you want to restrict the possible sources)
- Settings that specify whether to use the character set given in the inbound request, and if not, what other character set to use

The inbound HTTP adapter listens to a port on the local machine, reads the input, and sends the input as a stream to the associated business service. The business service, which you create and configure, uses this stream and communicates with the rest of the production. The following figure shows the overall flow:



In more detail:

1. The adapter receives an HTTP message and opens a TCP connection. (HTTP is a format of header and body data that is sent over a TCP connection.)
2. When the adapter connects, it executes its **OnConnected()** method, which first determines the character set to use. By default, it uses the character set specified in the inbound HTTP request. For details, however, see [Specifying the Character Set to Use](#).
3. The adapter chooses the appropriate translation table for the character set.
4. The adapter reads the body of the input, translates it, and places it into a new stream object.
 - If the adapter is using a non-binary character set, the stream is of type %GlobalCharacterStream.
 - If the adapter is using a binary character set, the stream is of type %GlobalBinaryStream.

Note that %GlobalCharacterStream and %GlobalBinaryStream are deprecated but are still supported for use in this way. It is not recommended to substitute different stream classes for this use case.

The adapter also extracts each HTTP header and adds that header to the Attributes property of the stream; this property is a multidimensional array, as discussed later.

Also, if the URL includes form parameters, these are passed as follows:

- If the HTTP request is a GET request, the adapter puts them into the Attributes array under the "Params" subscript.
 - If the HTTP request is a POST request, then [the form variables are written into the request body](#).
5. The adapter then calls the internal `ProcessInput()` method of the business service class, passing the stream as an input argument.
 6. The internal `ProcessInput()` method of the business service class executes. This method performs basic production tasks such as maintaining internal information as needed by all business services. You do not customize or override this method, which your business service class inherits.
 7. The `ProcessInput()` method then calls your custom `OnProcessInput()` method, passing the stream object as input. The requirements for this method are described later in [Implementing the OnProcessInput\(\) Method](#).

8. If `ProcessInput()` or `OnProcessInput()` returns an error, the production invokes the `OnErrorStream()` method of the business service.

The response message follows the same path, in reverse.

2.2 Creating a Business Service to Use the HTTP Inbound Adapter

To use this adapter in your production, create a new business service class as described here. Later, [add it to your production and configure it](#). You must also create appropriate message classes, if none yet exist. See [Defining Messages](#).

The following list describes the basic requirements of the business service class:

- Your business service class should extend `Ens.BusinessService`.
- In your class, the `ADAPTER` parameter should equal `EnsLib.HTTP.InboundAdapter`.
- If you need to parse form variables from an HTTP POST request, implement the **OnInit()** callback method as follows:

Class Member

```
Method OnInit() As %Status
{
    Set ..Adapter.ParseBodyFormVars=1
    Quit 1
}
```

This step is not necessary to parse the form variables from a GET request; these are automatically parsed into the `Attributes` property as [discussed below](#).

- Your class should implement the `OnProcessInput()` method, as described in [Implementing the OnProcessInput\(\) Method](#).
- Your class can implement the `OnErrorStream()` method. See [Implementing the OnErrorStream\(\) Method](#).
- For other options and general information, see [Defining a Business Service Class](#).

The following example shows the overall structure of your business service class. The details of the `OnProcessInput()` method depend on the character set that the adapter is using. If the adapter is using a non-binary character set, the general structure should be as follows:

Class Definition

```
Class EHTTP.NewService1 Extends Ens.BusinessService
{
    Parameter ADAPTER = "EnsLib.HTTP.InboundAdapter";

    Method OnProcessInput(pInput As %GlobalCharacterStream, pOutput As %RegisteredObject) As %Status
    {
        set tsc=$$0K
        //your code here
        Quit tsc
    }
}
```

Or, if the adapter is using a binary character set, the **OnProcessInput()** method should be as follows instead:

Class Member

```
Method OnProcessInput(pInput As %GlobalBinaryStream, pOutput As %RegisteredObject) As %Status
{
    set tsc=$$0K
    //your code here
    Quit tsc
}
```

2.3 Implementing the OnProcessInput() Method

Within your custom business service class, the signature of your **OnProcessInput()** method depends on the character set that the adapter is using:

- If the adapter is using a non-binary character set, the signature should be as follows:

```
Method OnProcessInput(pInput As %GlobalCharacterStream, pOutput As %RegisteredObject) As %Status
```

- If the adapter is using a binary character set, the signature should be as follows:

```
Method OnProcessInput(pInput As %GlobalBinaryStream, pOutput As %RegisteredObject) As %Status
```

Here *pInput* is the input that the adapter will send to this business service. Also, *pOutput* is the generic output argument required in the method signature.

Note that %GlobalCharacterStream and %GlobalBinaryStream are deprecated but are still supported for use in this way. It is not recommended to substitute different stream classes for this use case.

The **OnProcessInput()** method should do some or all of the following:

1. Examine the input object and extract the needed data from it. See [Using the HTTP Request](#).
2. Create an instance of the request message, which will be the message that your business service sends.

For information on creating message classes, see [Defining Messages](#).

3. For the request message, set its properties as appropriate, using values obtained from the input stream.

If the attribute Content - Type is specified in the Attributes property, by default, that is used as the Content - Type of the request message. If Content - Type is not specified in the Attributes property, by default, the Content - Type of the request message is set to "text/html". If these defaults are not appropriate, be sure to set this attribute. For example, the following code checks the value of the attribute Content - Type of the input stream and uses "text/xml" if the value is missing:

ObjectScript

```
Set outputContentType=$GET(pInput.Attributes("Content-Type"), "text/xml")
```

4. Call a suitable method of the business service to send the request to some destination within the production. Specifically, call **SendRequestSync()**, **SendRequestAsync()**, or (less common) **SendDeferredResponse()**. For details, see [Sending Request Messages](#).

Each of these methods returns a status (specifically, an instance of %Status).

5. Optionally check the status of the previous action and act upon it.
6. Optionally examine the response message that your business service has received and act upon it.
7. Make sure that you set the output argument (*pOutput*). This step is required.
8. Return an appropriate status. This step is required.

The following shows a simple example:

Class Member

```
Method OnProcessInput(pInput As %GlobalCharacterStream, Output pOutput As %RegisteredObject) As %Status
{
    //get contents of inbound stream
    //in this case, the stream contains a single value: a patient ID
    Set id=pInput.Read(, .tSC)

    //make sure Read went OK
    If $$$ISERR(tSC) do $System.Status.DisplayError(tSC)

    //create request object to send
    Set tRequest=##class(EHTTP.Request.Patient).%New()
    Set tRequest.patientID=id

    //send to lookup process
    Set tSC=..SendRequestSync("EHTTP.LookupProcess", tRequest, .tResponse)

    //define output for OnProcessInput
    Set pOutput=tResponse

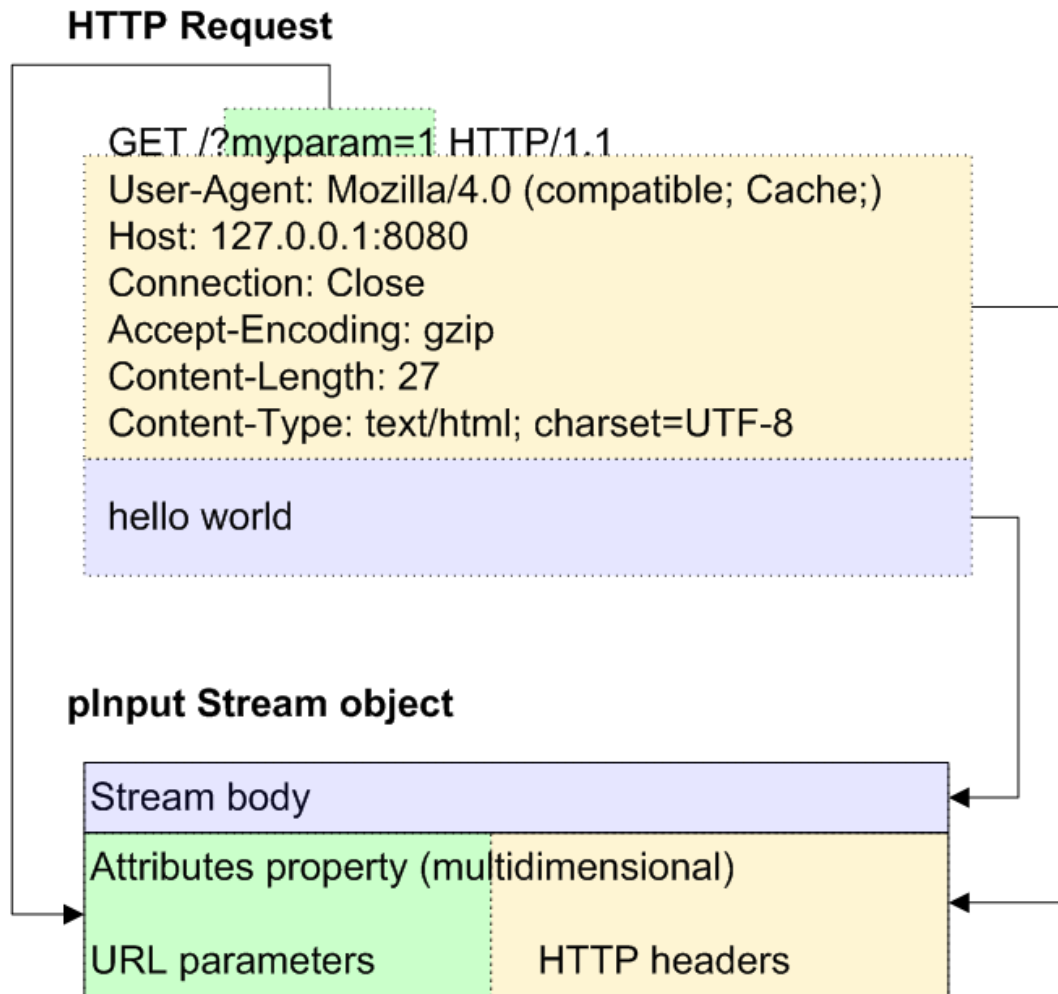
    Quit tSC
}
```

2.4 Implementing the OnErrorStream() Method

If the **ProcessInput()** or **OnProcessInput()** method of your business service returns an error, the production invokes the **OnErrorStream()** method of the business service. You can implement this method to contain any desired error handling. The method should accept a status code as input and should return the desired output stream.

2.5 Using the HTTP Request

Within **OnProcessInput()**, the HTTP request is available as *pInput*, which is an instance of %GlobalCharacterStream or %GlobalBinaryStream, depending on your implementation. The following figure illustrates how this instance represents the HTTP request:



The body of the request is written to the *pInput* stream. For information on working with streams, see [Working with Streams](#).

Additional data is available in the Attributes property of the *pInput* stream, as discussed in the following subsection. This includes the HTTP headers. If the request was a GET request, it also includes any form variables (URL parameters).

If the request was a POST request, then [the form variables are available in the body](#).

2.5.1 About the Attributes Array

The Attributes property of the *pInput* stream is a multidimensional array that carries the following data:

Node	Contents
Attributes(<i>http_header_name</i>) Where <i>http_header_name</i> is the header name in lowercase, for example, "content-length"	Value of the given HTTP header
Attributes("Params", <i>form_variable_name</i> , <i>n</i>)	Value of the <i>n</i> th instance of the given URL form variable (if the HTTP request was a GET request).
Attributes("URL")	Complete URL of the HTTP request

So you can retrieve a header value as follows:

ObjectScript

```
set contentlength = pInput.Attributes("content-length")
```

Or, to retrieve a URL form variable (for a GET request):

ObjectScript

```
set pResponse.MessageRequestTimeStamp = pInput.Attributes("Params","REQUESTTIMESTAMP",1)
```

2.5.2 Retrieving Form Variables from a POST Request

If the HTTP request was a POST request, then the form variables are available in the *pInput* stream. The form variables will be in key-value pairs, separated by an & delimiter. For example:

```
Q1=Answer1&Q2=Answer2&Q3=Answer3
```

Use ObjectScript string functions to retrieve the form variables. The following shows you an example of how to do this:

Class Member

```
Method OnProcessInput(pInput As %GlobalCharacterStream, pOutput As %RegisteredObject) As %Status
{
    Set tData=pInput.Read(, .tSC)
    If $$$ISERR(tSC) {
        Do $System.Status.DisplayError(tSC)
    }

    Set tRequest=##class(EHTTP.Request.Patient).%New()
    //use a delimiter to separate the form variables
    Set list=$LISTFROMSTRING(tData,"&")
    //examine each element and extract the relevant data
    Set ptr=0
    While $LISTNEXT(list,ptr,key) {
        If $PIECE(key,"=") = "Q1" {
            Set tRequest.Q1 = $PIECE(key,"=",2)
        }
        ElseIf $PIECE(key,"=") = "Q2" {
            SET tRequest.Q2 = $PIECE(key,"=",2)
        }
        ElseIf $PIECE(key,"=") = "Q3" {
            Set tRequest.Q3 = $PIECE(key,"=",2)
        }
    }

    Set tSC=..SendRequestSync("EHTTP.LookupProcess",tRequest, .tResponse)
    Set pOutput=tResponse
    Quit tSC
}
```

You can also use the [OnInit\(\)](#) to parse the form variables of a POST request.

2.6 Adding and Configuring the Business Service

To add your business service to a production, use the Management Portal to do the following:

1. Add an instance of your custom business service class to the production.
2. Configure the adapter so that it can receive input. Specifically:
 - Specify the port on which the adapter will listen. To do so, specify the [Port](#) setting.

- [Optionally specify the IP addresses](#) from which the adapter will accept input, if you want to limit the sources with which the adapter communicates.
 - [Optionally specify the character set](#) of the input data.
3. Enable the business service.
 4. Run the production.

2.6.1 Specifying the Sources of HTTP Requests

You can configure the inbound HTTP adapter to recognize sources of HTTP requests in two different ways:

- You can permit HTTP requests from any server. This is the default.
- You can permit HTTP requests from a list of specific servers (optionally with specific ports).

To do so, specify the [Allowed IP Addresses](#) setting. See [HTTP Adapter Settings](#).

2.6.2 Specifying the Character Set to Use

When the `EnsLib.HTTP.InboundAdapter` receives input, it translates the characters in that input according to a translation table. To determine which translation table to use, the adapter first determines which character set the input uses.

In general, the HTTP Content-Type header of the input indicates which character set that request uses. By default, the adapter uses that character set.

However, you can control which character set the adapter uses, by using the following runtime settings:

- [Force Charset](#)
- [Charset](#)

See [HTTP Adapter Settings](#).

For information on character sets and translation tables, see [Translation Tables](#).

2.7 See Also

- [About the HTTP Adapters](#)
- [Settings for the HTTP Inbound Adapter](#)
- [Business Host Classes That Use HTTP and SOAP Adapters](#)
- [Built-in HTTP Components](#)

3

Using the HTTP Outbound Adapter

This page describes the overall behavior of the HTTP outbound adapter (EnsLib.HTTP.OutboundAdapter) and describes how to use this adapter in your productions.

Tip: InterSystems IRIS® data platform also provides specialized business service classes that use this adapter, and one of those might be suitable for your needs. If so, no programming would be needed. See [Built-in HTTP Components](#) and [Business Host Classes That Use HTTP and SOAP Adapters](#).

3.1 Overall Behavior

Within a production, an outbound adapter is associated with a business operation that you create and configure. The business operation receives a message from within the production, looks up the message type, and executes the appropriate method. This method usually executes methods of the associated adapter.

EnsLib.HTTP.OutboundAdapter is the adapter for sending HTTP requests outside a production and receiving HTTP responses. This adapter provides settings to control items such as the following:

- Server and port to which the adapter will send HTTP requests
- URL path for the resource to request, at the given server and port
- An optional TLS configuration to use for the connection to the server
- Optional information to specify a proxy server through which the adapter can route requests

The HTTP requests are in the default character encoding of the local InterSystems IRIS server. To specify a different character encoding, create and send custom HTTP requests, as described in [Creating Custom HTTP Requests](#).

3.2 Creating a Business Operation to Use the Adapter

To create a business operation to use the EnsLib.HTTP.OutboundAdapter, you create a new business operation class. Later, [add it to your production and configure it](#).

You must also create appropriate [message classes](#), if none yet exist.

The following list describes the basic requirements of the business operation class:

- Your business operation class should extend Ens.BusinessOperation.

- In your class, the *ADAPTER* parameter should equal `EnsLib.HTTP.OutboundAdapter`.
- In your class, the *INVOCATION* parameter should specify the invocation style you want to use, which must be one of the following.
 - **Queue** means the message is created within one background job and placed on a queue, at which time the original job is released. Later, when the message is processed, a different background job is allocated for the task. This is the most common setting.
 - **InProc** means the message will be formulated, sent, and delivered in the same job in which it was created. The job will not be released to the sender's pool until the message is delivered to the target. This is only suitable for special cases.
- Your class should define a *message map* that includes at least one entry. A message map is an XData block entry that has the following structure:

```
XData MessageMap
{
  <MapItems>
    <MapItem MessageType="messageclass">
      <Method>methodname</Method>
    </MapItem>
    ...
  </MapItems>
}
```

- Your class should define all the methods named in the message map. These methods are known as *message handlers*. Each message handler should have the following signature:

```
Method Sample(pReq As RequestClass, Output pResp As ResponseClass) As %Status
```

Here *Sample* is the name of the method, *RequestClass* is the name of a request message class, and *ResponseClass* is the name of a response message class. In general, the method code will refer to properties and methods of the Adapter property of your business operation.

For information on defining message classes, see [Defining Messages](#).

For information on defining the message handler methods, see [Creating Message Handler Methods](#)

- For other options and general information, see [Defining a Business Operation Class](#).

The following example shows the general structure that you need:

Class Definition

```
Class EHTTP.NewOperation1 Extends Ens.BusinessOperation
{
  Parameter ADAPTER = "EnsLib.HTTP.OutboundAdapter";
  Parameter INVOCATION = "Queue";
  Method Sample(pReq As RequestClass, Output pResp As ResponseClass) As %Status
  {
    Quit $$$ERROR($$$NotImplemented)
  }
  XData MessageMap
  {
    <MapItems>
      <MapItem MessageType="RequestClass">
        <Method>Sample</Method>
      </MapItem>
    </MapItems>
  }
}
```

3.3 Creating Message Handlers

When you create a business operation class for use with `EnsLib.HTTP.OutboundAdapter`, typically your biggest task is writing message handlers for use with this adapter, that is, methods that receive production messages and then perform various HTTP operations.

Each message handler method should have the following signature:

```
Method Sample(pReq As RequestClass, Output pResp As ResponseClass) As %Status
```

Here *Sample* is the name of the method, *RequestClass* is the name of a request message class, and *ResponseClass* is the name of a response message class.

In general, the method should do the following:

1. Examine the inbound request message.
2. Using the information from the inbound request, call a method of the `Adapter` property of your business operation. For example, call the **Get** method, which sends the HTTP GET command:

```
set status=..Adapter.Get(.pHttpResponse,"Name",pRequest.Name)
```

This example sends a GET request, passing in a parameter called `Name`, which has the value of `pRequest.Name`.

The available methods are discussed in [the next section](#). Each of them returns a status (specifically an instance of `%Status`).

These methods also return an HTTP response as output. In the preceding example, the response is placed into `pHttpResponse`. The response is an instance of `%Net.HttpResponse`; for information on using this object, see [Using the HTTP Response](#).

3. Examine the HTTP response.
4. Use information in the HTTP response to create a response message (an instance of `Ens.Response` or a subclass), which the method returns as output.

For basic information on defining message classes, see [Defining Messages](#).

5. Make sure that you set the output argument (`pOutput`). Typically you set this equal to the response message. This step is required.
6. Return an appropriate status. This step is required.

The following shows an example:

Class Member

```
Method PostMessage(pRequest As EHTTP.Request.OutboundPost,
Output pResponse As EHTTP.Response.OutboundPost) As %Status
{
  Set tSC=$$OK
  Set input=pRequest.MessageStream
  Set tResponse = ##class(%Net.HttpResponse).%New()
  Set tSC = ..Adapter.Post(.tResponse,,input)

  If $$$ISOK(tSC){
    Set pResponse = ##class(EHTTP.Response.OutboundPost).%New()
    Set len = tResponse.Data.SizeGet()
    While (tResponse.Data.AtEnd = 0) {
      Do pResponse.MessageStream.Write(tResponse.Data.Read())
    }
  }
  Return tSC
}
```

3.4 Calling HTTP Commands

The adapter provides methods for sending HTTP POST, GET, PATCH, PUT and DELETE requests:

- The main methods are named after the HTTP action. For example, the `Post()` method handles POST requests. Each of these main methods accepts an output argument for the response object, a comma-separated list of form variable names, and a variable number of form variable arguments, one for each of the names in the comma-separated list. If you want to set multiple values for a form variable, you may use the same name multiple times in the list. Of course, you can also use these methods with no form variables to request flat scalar content such as a regular web page.
- For situations with a complicated set of form variables, you can use alternate methods that accept a multidimensional array instead of a variable argument list. Each HTTP action has one of these alternate methods. For example, instead of the `Post()` method you could use the `PostFormDataArray()` method. This can help keep things organized, because multiple values for a given form variable can be given as subnodes in the array, rather than by using multiple entries in the list of names. You can also index the array by form variable name rather than by position.
- The low-level worker method `SendFormDataArray()` is available for situations when you need to use some other unusual HTTP request, or where you need to customize aspects of your HTTP request other than form variables or cookies.

The following sections provide more information about using methods of `EnsLib.HTTP.OutboundAdapter` to send HTTP commands:

- [How to send form data](#)
- [How to send a request body](#)
- [Reference information for the adapter methods](#)
- [Basic information about the HTTP response](#)

3.4.1 Sending Form Data

You can send HTTP form data using any of the adapter's methods. Each method returns (as output) an HTTP response, which is an instance of `%Net.HttpResponse`. For information on using this object, see [Using the HTTP Response](#).

The details differ from method to method, but a partial example follows:

```
set tFormVar="USER,ROLE,PERIOD"

set tUserID=pRequest.UserID
set tRole=pRequest.UserRole
set tED=pRequest.EffectiveDate

set tSC=..Adapter.Get(.tResponse,tFormVar,tUserID,tRole,tED)
```

Note that this example assumes that the request message has the properties `UserID`, `UserRole`, and `EffectiveDate`.

3.4.2 Sending a Request Body

You can send a request body using any of the adapter's methods. In this case, you will pass the request body as an argument, and you will leave the form data argument empty. A request body can be either a stream or a string, depending on your needs. For information on creating a stream and writing to it, see [Working with Streams](#).

Each of these methods returns (as output) an HTTP response, an instance of `%Net.HttpResponse`. For information on using this object, see [Using the HTTP Response](#).

The details differ from method to method, but a partial example follows:

```
set tsc = ..Adapter.Post(.tResponse,,pRequest.MessageStream)
```

Note that this example assumes that the request message has a property `MessageStream`.

Additionally, note that to use the `EnsLib.HTTP.OutboundAdapter` to POST data formatted as text/xml, such data must be placed in the `EntityBody` property of the `HTTPRequest` object that will be entered as an input to the class method you will use to POST data.

3.4.2.1 Stream Attributes as HTTP Headers

If you create a stream and send that as the request body, you can force the HTTP outbound adapter to include some or all of the stream attributes as HTTP headers. To do so, set the `SkipBodyAttrs` property equal to a list of attributes that should *not* be used as HTTP headers. The default value is "*" which means that the stream attributes are ignored by default (and not used as headers). Note that this property is not available as a runtime setting.

For information on stream attributes, see the class reference for `%Library.AbstractStream`.

3.4.3 Reference Information for HTTP Methods

This section provides reference information on the methods you can use to invoke HTTP commands.

Post()

```
Method Post(Output pHttpResponse As %Net.HttpResponse,
            pFormVarNames As %String = "",
            pData...) As %Status
```

Sends the HTTP POST command to the configured destination (see [Specifying the Destination](#)), sending either form data or a request body.

The HTTP response is returned as output in the first argument. This is an instance of `%Net.HttpResponse`; for information on using this object, see [Using the HTTP Response](#).

With this method, do one of the following:

- To send form data to the named form variables, specify the `pFormVarNames` argument and `pData` arguments as needed. `pFormVarNames` is a comma-separated list of form variable names to use. For each name in the list, a `pData` argument should be supplied.

To pass multiple values for a given form variable, include the variable name multiple times in the `pFormVarNames` list.

Any extra `pData` arguments you supply will be assigned to the last form variable in the list.

- To send a request body instead of form variables, leave `pFormVarNames` empty and pass the body text as the `pData` argument.

A data value passed for body text may be of either string or stream type. Data values passed for form variables must be of string type.

PostFormDataArray()

```
Method PostFormDataArray(Output pHttpResponse As %Net.HttpResponse,
                        pFormVarNames As %String = "",
                        ByRef pData) As %Status
```

Sends the HTTP POST command to the configured destination (see [Specifying the Destination](#)), sending form data to the named form variables.

pFormVarNames is a comma-separated list of form variable names to use. For each name in the list, a *pData* argument should be supplied.

The *pData* argument is an array. The top node of the array is not used. Each subnode is subscripted by the index of the corresponding form variable in the *pFormVarNames* list. The value at a given subscript should be specified as follows:

- For a form variable (*varname*) with a single value, the value at *pData*("varname") should be the form data value to send. There should be no subnodes.
- For a form variable (*varname*) with multiple values, the value *pData*("varname") should be the count of the values. Each of the values for this form variable should appear in a subnode, subscripted by its position in the node.

PostURL()

```
method PostURL(pURL As %String,  
               Output pHttpResponse As %Net.HttpResponse,  
               pFormVarNames As %String = "",  
               pData...) as %Status
```

Sends the HTTP POST command to the specified URL (*pURL*), sending form data to the named form variables. This allows you to override the adapter's URL property. See the remarks for **Post()**.

Get()

```
Method Get(Output pHttpResponse As %Net.HttpResponse,  
           pFormVarNames As %String = "",  
           pData...) As %Status
```

Sends the HTTP GET command to the configured destination (see [Specifying the Destination](#)), sending either form data or a request body. See the remarks for **Post()**.

GetFormDataArray()

```
Method GetFormDataArray(Output pHttpResponse As %Net.HttpResponse,  
                        pFormVarNames As %String = "",  
                        ByRef pData) As %Status
```

Sends the HTTP GET command to the configured destination (see [Specifying the Destination](#)), sending form data to the named form variables. See the remarks for **PostFormDataArray()**.

GetURL()

```
method GetURL(pURL As %String,  
              Output pHttpResponse As %Net.HttpResponse,  
              pFormVarNames As %String = "",  
              pData...) as %Status
```

Sends the HTTP GET command to the specified URL (*pURL*), sending form data to the named form variables. This allows you to override the adapter's URL property. See the remarks for **Post()**.

Put()

```
Method Put(Output pHttpResponse As %Net.HttpResponse,  
           pFormVarNames As %String = "",  
           pData...) As %Status
```

Sends the HTTP PUT command to the configured destination (see [Specifying the Destination](#)), sending either form data or a request body. See the remarks for **Post()**.

PutFormDataArray()

```
Method PutFormDataArray(Output pHttpResponse As %Net.HttpResponse,
                        pFormVarNames As %String = "",
                        ByRef pData) As %Status
```

Sends the HTTP PUT command to the configured destination (see [Specifying the Destination](#)), sending form data to the named form variables. See the remarks for **PostFormDataArray()**.

PutURL()

```
method PutURL(pURL As %String,
              Output pHttpResponse As %Net.HttpResponse,
              pFormVarNames As %String = "",
              pData...) as %Status
```

Sends the HTTP PUT command to the specified URL (*pURL*), sending form data to the named form variables. This allows you to override the adapter's URL property. See the remarks for **Post()**.

Patch()

```
Method Patch(Output pHttpResponse As %Net.HttpResponse,
             pFormVarNames As %String = "",
             pData...) As %Status
```

Sends the HTTP PATCH command to the configured destination (see [“Specifying the Destination”](#)), sending either form data or a request body. See the remarks for **Post()**.

PatchFormDataArray()

```
Method PatchFormDataArray(Output pHttpResponse As %Net.HttpResponse,
                          pFormVarNames As %String = "",
                          ByRef pData) As %Status
```

Sends the HTTP PATCH command to the configured destination (see [“Specifying the Destination”](#)), sending form data to the named form variables. See the remarks for **PostFormDataArray()**.

PatchURL()

```
method PatchURL(pURL As %String,
                Output pHttpResponse As %Net.HttpResponse,
                pFormVarNames As %String = "",
                pData...) as %Status
```

Sends the HTTP PATCH command to the specified URL (*pURL*), sending form data to the named form variables. This allows you to override the adapter's URL property. See the remarks for **Post()**.

Delete()

```
method Delete(Output pHttpResponse As %Net.HttpResponse,
              pFormVarNames As %String = "",
              pData...) as %Status
```

Sends the HTTP DELETE command to the configured destination (see [Specifying the Destination](#)), sending either form data or a request body. See the remarks for **Post()**.

DeleteFormDataArray()

```
method DeleteFormDataArray(Output pHttpResponse As %Net.HttpResponse,
                           pFormVarNames As %String = "",
                           ByRef pData) as %Status
```

Sends the HTTP DELETE command to the configured destination (see [Specifying the Destination](#)), sending form data to the named form variables. See the remarks for **PostFormDataArray()**.

DeleteURL()

```
method DeleteURL(pURL As %String,  
                Output pHttpResponse As %Net.HttpResponse,  
                pFormVarNames As %String = "",  
                pData...) as %Status
```

Sends the HTTP DELETE command to the specified URL (*pURL*), sending form data to the named form variables. This allows you to override the adapter's URL property. See the remarks for **Post()**.

SendFormDataArray()

```
Method SendFormDataArray(Output pHttpResponse As %Net.HttpResponse,  
                        pop As %String,  
                        pHttpRequestIn As %Net.HttpRequest,  
                        pFormVarNames As %String = "",  
                        ByRef pData) As %Status
```

Sends the HTTP request to the configured destination (see [Specifying the Destination](#)), sending form data to the named form variables.

The *pOp* argument specifies the HTTP action to take. This should be one of the following: "POST" "GET" "PUT"

pFormVarNames is a comma-separated list of form variable names to use. For each name in the list, a *pData* argument should be supplied.

The *pData* argument is an array; see the remarks for the **GetFormDataArray()** method.

For special needs, create an instance of %Net.HttpRequest or a subclass, specify its properties, and use this as the *pHttpRequestIn* argument. If you do this, the HTTP request is initialized with the properties of your instance. This method gives you more control than the other methods of the outbound adapter. For example, it enables you to add your own HTTP headers to the request.

3.4.4 Handling the HTTP Response

When you use any of the methods described in the previous subsection, you receive, as output, an HTTP response. This object is an instance of %Net.HttpResponse. For more details, see [Using the HTTP Response](#).

3.5 Managing Cookies

A cookie is a text string in a response header that a server can ask a client to save and return as a header value in subsequent HTTP requests. Some servers use cookies to maintain open sessions,

The EnsLib.HTTP.OutboundAdapter provides the following properties and methods to manage cookies. Your custom methods can use these.

UseCookies property

Specifies whether to save cookies received in HTTP responses while this adapter is instantiated, and insert them in each subsequent HTTP request. If UseCookies is true, then any job associated with an instance of that adapter will maintain a collection of cookies and send the appropriate selection of them with each new request that it sends. If UseCookies is false (0), cookies will not be sent. This convention allows each of these jobs to maintain its own persistent session with any web server that requires it.

The default is false. This property is also available as a runtime setting.

%Cookies property

Contains an array of cookies. Indexed by Domain/Server; each element is \$LB(name, domain, path, value, expires, secure). This property is InterSystems IRIS multidimensional array.

DeleteCookie() method

```
Method DeleteCookie(pName As %String,
                   pPath As %String,
                   pDomain As %String) As %Status
```

Deletes a particular cookie.

DeleteCookies() method

```
Method DeleteCookies(pDomain As %String = "",
                    pPath As %String = "") As %Status
```

Deletes all cookies from the specified domain and/or path.

Note: Remember that cookies are specific to an HTTP server. When you insert a cookie, you are using a connection to a specific server, and the cookie is not available on other servers.

3.6 Creating Custom HTTP Requests

If you use the more common methods of the HTTP outbound adapter (such as **Get**), the adapter automatically creates and sends an HTTP request, which can include either form data or a request body. In special cases, you may want to create a custom HTTP request so that you can specify details such as proxy authorization or a different character encoding.

To send the custom request, use the **SendFormDataArray()** method of the HTTP outbound adapter as described in [Calling HTTP Commands](#). Then see Sending an HTTP Request.

3.7 Using the HTTP Response

After you use the outbound adapter to send an HTTP request, you receive a response object (%Net.HttpResponse). Specifically, the main methods of the HTTP.OutboundAdapter enable you to send an HTTP request. All these methods give you access to the HTTP response that is sent in return:

- If you use the **PostFormData()**, **PostFormDataArray()**, **GetFormData()** or **GetFormDataArray** method, the HTTP response is returned as output in the first argument of the method call. This argument is an instance of %Net.HttpResponse.
- If you use the **SendFormDataArray()** method, the HttpResponse property of the request is updated. This property is an instance of %Net.HttpResponse.

For details on these methods, see [Calling HTTP Commands](#).

For more information on using the HTTP response, see Using an HTTP Response.

3.8 Examples

This section provides a couple of examples.

3.8.1 Example with Post

The following example uses the HTTP outbound adapter and posts an XML message to the configured destination. First, the business operation class is as follows:

Class Definition

```
Class EHTTP.PostOperation Extends Ens.BusinessOperation
{
    Parameter ADAPTER = "EnsLib.HTTP.OutboundAdapter";
    Parameter INVOCATION = "Queue";

    Method PostMessage(pRequest As EHTTP.Request.OutboundPost,
    Output pResponse As EHTTP.Response.OutboundPost) As %Status
    {
        Set tSC=$$$$OK

        Set tResponse = ##class(%Net.HttpResponse).%New()
        Set tSC = ..Adapter.Post(.tResponse,,pRequest.MessageStream)
        Set stream = ""

        If $$$ISOK(tSC){
            Set pResponse = ##class(EHTTP.Response.OutboundPost).%New()
            Set len = tResponse.Data.SizeGet()
            While (tResponse.Data.AtEnd = 0) {
                Do pResponse.MessageStream.Write(tResponse.Data.Read())
            }
        }
        Return tSC
    }
}

XData MessageMap
{
    <MapItems>
        <MapItem MessageType="EHTTP.Request.OutboundPost">
            <Method>PostMessage</Method>
        </MapItem>
    </MapItems>
}
}
```

The request message class is as follows:

Class Definition

```
Class EHTTP.Request.OutboundPost Extends Ens.Request
{
    /// MessageStream contains the complete SOAP Message to post
    Property MessageStream As %GlobalCharacterStream(CONTENT = "MIXED");
}
}
```

The response message class is as follows:

Class Definition

```
Class EHTTP.Response.OutboundPost Extends Ens.Response
{
    /// MessageStream contains the Response to the SOAP Message post
    Property MessageStream As %GlobalCharacterStream(CONTENT = "MIXED");
}
}
```

3.8.2 Example with Get

The following example uses the **Get()** method.

```
Method GetEvent(pRequest As EHTTP.Request.GetEvent,
ByRef pResponse As EHTTP.Response.GetEvent) As %Status
{
  Set tSC=$$$OK

  Set pResponse=##class(EHTTP.Response.GetEvent).%New()

  Set tFormVar="Name, Country, City"

  Set tName=pRequest.EventName
  Set tCountry=pRequest.EventCountry
  Set tCity=pRequest.EventCity

  Set tSC=..Adapter.Get(.tResponse, tFormVar, tName, tCountry, tCity)

  If '$IsObject(tResponse) {Quit
  }
  ;
  ;now parse the XML stream
  ;
  Set reader=(%XML.Reader).%New()
  Do tResponse.Data.Rewind()
  Set tSC=reader.OpenStream(tResponse.Data)

  If $$$ISERR(tSC) {
    $$$LOGWARNING("Unable to open the XML Stream")
    $$$TRACE("XML request probably failed")
    Do tResponse.Data.Rewind()
    Set traceline = tResponse.Data.Read(1000)
    $$$TRACE(traceline)
    Set tSC=$$$OK
    Quit
  }
  ;
  ;Associate a class name with the XML element Name
  ;
  Do reader.Correlate("EventResult", "EHTTP.Event.EventResult")

  If reader.Next(.tResults, .tSC)
  {
    Set pResponse.EventValues=tResults.EventValues
  }
  Return tSC
}
```

This example retrieves an XML file from the target server, parses it, retrieving specific data.

3.9 Adding and Configuring the Business Operation

To add your business operation to a production, use the Management Portal to do the following:

1. Add an instance of your custom business operation class to the production.
2. Configure the adapter to communicate with a specific external data source. Specifically:
 - [Specify the destination for HTTP requests](#)
 - [Optionally specify a proxy server](#)
3. Enable the business operation.
4. Run the production.

3.9.1 Specifying the Destination Server and URL Path

Use the following runtime settings to specify the destination to which you will send HTTP requests:

- [HTTP Server](#)
- [HTTP Port](#)
- [Credentials](#)
- [URL](#)

3.9.1.1 Example

For example, suppose that the URL to which you are posting is `http://122.67.980.43/HTTPReceive.aspx`

In this case, you would use the following settings:

Setting	Value
HTTPServer	122.67.980.43
URL	HTTPReceive.aspx

3.9.2 Specifying a Proxy Server

Use the following runtime settings to route the HTTP request via a proxy server, if needed:

- [Proxy Server](#)
- [Proxy Port](#)
- [Proxy HTTPS](#)

3.10 See Also

- [About the HTTP Adapters](#)
- [Calling HTTP Commands](#)
- [Sending an HTTP Request](#)
- [Using an HTTP Response](#)
- [Creating Custom HTTP Requests](#)
- [Settings for the HTTP Outbound Adapter](#)
- [Business Host Classes That Use HTTP and SOAP Adapters](#)
- [Built-in HTTP Components](#)

4

Built-in HTTP Components

InterSystems IRIS® data platform provides built-in business hosts that use the HTTP adapters, allowing you to add HTTP support to a production without creating a custom business service or business operation. This page briefly introduces them, as well as the generic HTTP message class.

4.1 Available HTTP Business Hosts

If you need a business service that uses the HTTP inbound adapter, you can add `EnsLib.HTTP.GenericService` to the production. This business service uses `EnsLib.HTTP.InboundAdapter` as its adapter.

Similarly, a production that needs to leverage the HTTP outbound adapter can use `EnsLib.HTTP.GenericOperation`. This business operation uses `EnsLib.HTTP.OutboundAdapter` as its adapter.

4.2 Generic HTTP Message Class

InterSystems IRIS provides a message class designed to carry a HTTP request or response through a production. This message class, `EnsLib.HTTP.GenericMessage`, includes the headers and body of the HTTP request. In some cases, it may be useful to construct a `EnsLib.HTTP.GenericMessage` from scratch and then send it out using `EnsLib.HTTP.GenericOperation`. This process consists of building the body and populating the headers before creating the new message. The following is an example:

```
// Build the header
#dim tRESTHTTPHeaders
Set tRESTHTTPHeaders("HttpRequest")="POST"
Set tRESTHTTPHeaders("HTTPVersion")="1.1"

// Build the body
#dim tPOSTStream = ##class(%Stream.GlobalCharacter).%New()
#dim tPOSTJSON = {}
Set tPOSTJSON.projection = []
Do tPOSTJSON.projection.%Push("%Doc")
Do tPOSTJSON.%ToJSON(.tPOSTStream)

// Add more headers
Set tRESTHTTPHeaders("content-length") = tPOSTStream.Size
Set tRESTHTTPHeaders("content-type") = "application/json"

// Create message
Return ##class(EnsLib.HTTP.GenericMessage).%New(tPOSTStream,,.tRESTHTTPHeaders)
```

When creating the HTTP message using `EnsLib.HTTP.GenericMessage.%New()`, you have several options for passing in the headers. Commonly, the header argument is passed in as an array. However, the **%New()** method also accepts headers as an `ObjectScript` array reference, an `%AbstractStream` with its own `Attributes` to copy from, an `%Net.HttpResponse` object with its own `Headers` to copy from, or a string of the form 'a=1, b=2, c=3'.

4.3 Target URL of the HTTP Request

When a production uses `EnsLib.HTTP.GenericOperation`, the URL header of the message is processed in conjunction with the URL property of the outbound adapter to determine the URL path of the request. Like other adapter properties, the value of this URL property is determined by the **URL** setting of the production's business operation. For details on how the URL header and the URL property interact, refer to the class reference for the URL property of `EnsLib.HTTP.OutboundAdapter`.

4.4 URL Parameters

By default, URL parameters are sent in the HTTP request body. To force URL parameters to be sent in the URL, set the `RawParams` HTTP header. (If the `IParams` header is specified, then that will be used if the `RawParams` header is empty.)

4.5 More About Message Headers

The following example shows what the headers of a `EnsLib.HTTP.GenericMessage` might look like if it were created by InterSystems IRIS:

```
<HTTPHeaders>
  <HTTPHeadersItem HTTPHeadersKey="CharEncoding" xsi:nil="true"></HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="EnsConfigName">ForDocker</HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="HTTPVersion">1.1</HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="HttpRequest">POST</HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="IParams">0</HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="RawParams" xsi:nil="true"></HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="TranslationTable">RAW</HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="URL">/api/atelier/v4/TEST/action/query</HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="accept">application/json, text/plain, */*</HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="connection">close</HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="content-length">127</HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="content-type">application/json</HTTPHeadersItem>
  <HTTPHeadersItem
HTTPHeadersKey="cookie">CSPSESSIONID-SP-42773-UP-api-atelier--001000000000t7LT7VX9SXUNIM7S8yKxK44SSuBbUtzNHVjIAK</HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="host">127.0.0.1:55773</HTTPHeadersItem>
  <HTTPHeadersItem HTTPHeadersKey="user-agent">axios/0.21.1</HTTPHeadersItem>
</HTTPHeaders>
```

If you are building a `EnsLib.HTTP.GenericMessage` from scratch, not all of these headers are required. At a minimum, the `HttpRequest`, `HTTPVersion`, and `content-type` headers should be defined, and the `content-length` header is also commonly defined.

4.6 See Also

- [About the HTTP Adapters](#)

- [Business Host Classes That Use HTTP and SOAP Adapters](#)
- [Using the HTTP Inbound Adapter](#)
- [Using the HTTP Outbound Adapter](#)
- [HTTP Adapter Settings](#)

HTTP Adapter Settings

This section provides reference information for the HTTP adapters.

Also see [Settings in All Productions](#).

Settings for the HTTP Inbound Adapter

Provides reference information for settings of the HTTP inbound adapter, `EnsLib.HTTP.InboundAdapter`. You can [configure](#) these settings after you have [added](#) a business service that uses [this adapter](#) to your production.

Summary

The inbound HTTP adapter has the following settings:

Group	Settings
Basic Settings	Call Interval , Port
Connection Settings	Job Per Connection , Allowed IP Addresses , OS Accept Connection Queue Size , Stay Connected , Read Timeout , SSL Configuration , Local Interface , Enable Standard Requests
Additional Settings	Charset , Force Charset , GenerateSuperSessionID

The remaining settings are common to all business services. For information, see [Settings for All Business Services](#).

Also, it is possible to limit the number of jobs instance-wide in order to avoid too many jobs being created by calls to an HTTP business service. To specify this limit, set `^%SYS("Ensemble", "MaxJobsForCreateBusinessService")` equal to the maximum desired number.

Allowed IP Addresses

Specifies a comma-separated list of remote IP addresses from which to accept connections. The adapter accepts IP addresses in dotted decimal form.

Note: IP address filtering is a means to control access on private networks, rather than for publicly accessible systems. InterSystems does not recommend relying on IP address filtering as a sole security mechanism, as it is possible for attackers to spoof IP addresses.

An optional `:port` designation is supported, so either of the following address formats is acceptable: `192.168.1.22` or `192.168.1.22:3298`. If a port number is specified, connections from other ports will be refused.

If the string starts with an exclamation point (!) character, the inbound adapter initiates the connection rather than waiting for an incoming connection request. The inbound adapter initiates the connection to the specified address and then waits for a message. In this case, only one address may be given, and if a port is specified, it supersedes the value of the `Port` setting; otherwise, the `Port` setting is used.

Also see [Specifying the Sources of HTTP Requests](#).

Call Interval

This adapter does not use polling. It listens for a connection, and once a connection is established, it listens for messages on that connection (and responds immediately, if possible). It does, however, periodically check to see whether there have been requests to shut down the adapter or make it quiescent. This setting specifies the interval, in seconds, for the `EnsLib.HTTP.InboundAdapter` to perform this check.

The default value is 5 seconds. The minimum is 0.1 seconds.

Charset

Specifies the character set of the incoming data. InterSystems IRIS® automatically translates from this encoding. The setting value is not case-sensitive. Use `Binary` for binary files, or for any data in which newline and line feed characters are distinct

or must remain unchanged, for example in HL7 Version 2 or EDI messages. Other settings may be useful when transferring text documents. Choices include:

- **Auto**—Use the encoding declared in the incoming HTTP header **Content - Type** field. This is the default.
- **AutoXML**—Use the encoding declared in the XML header of the incoming XML body content, if any.
- **Binary**—Read the raw bytes of the body without performing any character encoding transformation.
- **RawBytes**—Read the raw bytes of the body without performing any character encoding transformation.
- **Default**—Use the default character encoding of the local InterSystems IRIS server.
- **Latin1**—The ISO Latin1 8-bit encoding.
- **ISO-8859-1**—The ISO Latin1 8-bit encoding.
- **UTF-8**—The Unicode 8-bit encoding.
- **UCS2**—The Unicode 16-bit encoding.
- **UCS2-BE**—The Unicode 16-bit encoding in big-endian form.
- Any other alias from an international character encoding standard for which NLS (National Language Support) is installed in InterSystems IRIS.

For information on character sets and translation tables, see [Translation Tables](#).

Force Charset

If this setting is true, the adapter uses the **Charset** setting instead of any character set declared in the incoming HTTP header **Content - Type** field. The default is false.

Generate SuperSession ID

This setting controls whether the message will have a **SuperSessionID**, which can be used to identify messages that cross from one production to another. For details, see [Supersessions](#).

Job Per Connection

If this setting is true, the adapter spawns a new job to handle each incoming TCP connection and allows simultaneous handling of multiple connections. If it is false, the adapter does not spawn a new job for each connection. The default is true.

Local Interface

Specifies the network interface through which the connection should go. Select a value from the list or type a value. An empty value means use any interface.

Enable Standard Requests

Specifies that the business service associated with the adapter can receive requests from both the Web Gateway port and the [custom port](#) that you specify.

This setting is applicable only when the HTTP business service is based on the `EnsLib.HTTP.Service` class. For more information, see [Creating a Business Service to Use the HTTP Inbound Adapter](#).

The default value for HL7 HTTP business services is `false`. The default value for all other HTTP business services is `true`.

When you enable this setting, you must take additional steps to ensure that the Web Gateway can communicate with the business service:

- If only one business service is based on the custom class, set the [configuration name](#) of the business service to the name of the custom class. Then, include the name of the custom class in any URL that invokes the business service, for example:
`/csp/myapplication/mynamespace/myserviceclassname`
- If multiple business services are based on the custom class, include the `?CfgItem=confiitemname` parameter in any URL that invokes the business service, where *confiitemname* is the business service configuration name, for example:
`/csp/myapplication/mynamespace/myserviceclassname?CfgItem=myserviceconfignameA`
`/csp/myapplication/mynamespace/myserviceclassname?CfgItem=myserviceconfignameB`
- Additionally, if you use a CSP web application to communicate with the business service, set the **Dispatch Class** for the web application to the name of the custom class.

If you select **Enable Standard Requests** and set the pool size to 0, then business service receives data only from the Web Gateway port and not from the custom port.

Note: If you use the Web Gateway port, First-In-First-Out processing for messages is not supported.

OS Accept Connection Queue Size

Specifies the number of incoming connections should the operating system should hold open. Set to 0 if only one connection at a time is expected. Set to a large number if many clients will connecting rapidly.

Port

Identifies the TCP port on the local machine where the adapter is listening for HTTP requests. Avoid specifying a port number that is in the range used by the operating system for ephemeral outbound connections.

Read Timeout

Number of seconds to wait for each successive incoming read, following receipt of initial data from remote port.

Stay Connected

Specifies whether to keep the TCP connection open between requests.

- If this setting is zero, the adapter will disconnect immediately after each message is received.
- If this setting is positive, it specifies the idle time, in seconds. The adapter disconnects after this idle time.
- If this setting is -1, the adapter auto-connects on startup and then stays connected.

Settings for the HTTP Outbound Adapter

Provides reference information for settings of the HTTP outbound adapter, `EnsLib.HTTP.OutboundAdapter`. You can [configure](#) these settings after you have [added](#) a business operation that uses [this adapter](#) to your production.

Summary

The outbound HTTP adapter has the following settings:

Group	Settings
Basic Settings	HTTP Server , HTTP Port , URL , Credentials
Connection Settings	Backup HTTP Server , Backup HTTP Port , Retries To Failover , SSL Configuration , SSL Check Server Identity , Proxy Server , Proxy Port , Proxy HTTPS , Proxy Http Tunnel , Response Timeout , ConnectTimeout , WriteTimeout , LocalInterface
Additional Settings	Use Cookies , SendSuperSession

The remaining settings are common to all business operations. For information, see [Settings for All Business Operations](#).

Backup HTTP Port

TCP port on the backup server to send HTTP requests to. Leave this blank to use the default, which is 80 (or 443 if `SSLConfig` is specified).

Backup HTTP Server

IP address of the backup HTTP server, if any. This server must accept the same values as the original server for settings such as **Credentials** and **File Path**.

Connect Timeout

Specifies the number of seconds to wait for the connection to the server to open. The default value is 5.

If the connection is not opened in this time period, the adapter retries repeatedly, up to the number of times given by `Failure Timeout` divided by `Retry Interval`.

Credentials

ID of the production credentials that can authorize a connection to the given destination URL. See [Defining Production Credentials](#).

HTTP Port

TCP port on the server to send HTTP requests to (will use 80 by default, or 443 if `SSLConfig` is specified). Values other than 80 are included in the `Host :` header of the HTTP request that you are sending. For more information, see [Specifying the Destination Server and URL Path](#).

HTTP Server

IP address of the server to send HTTP requests to. This is used in the `Host :` header of the HTTP request that you are sending. Also see [Specifying the Destination Server and URL Path](#).

Local Interface

Specifies the network interface through which the HTTP connection should go. Select a value from the list or type a value. An empty value means use any interface.

Proxy HTTPS

If using a proxy server, indicates whether the proxy server communicates with the target system using HTTPS. If you enable this option and specify a value in the [Proxy Server](#) property, then the proxy server issues requests for HTTPS pages rather than HTTP pages. Additionally, enabling this option changes the default [HTTP Port](#) to 443, which is the HTTPS port.

Proxy Port

If using a proxy server, specifies the port on the proxy server to connect to. The default value is 8080.

Proxy Server

If using a proxy server, specifies the proxy server host name. Requests are made to the proxy server that you specify on the [Proxy Port](#) that you specify. If you do not specify a value, a proxy server is not used.

Proxy HTTP Tunnel

Specifies whether the adapter uses the HTTP CONNECT command to establish a tunnel through the proxy to the target HTTP server. If true, the request uses the HTTP CONNECT command to establish a tunnel. The address of the proxy server is taken from the Proxy Server and Proxy Port properties. If [Proxy Https](#) is true, then once the tunnel is established, InterSystems IRIS® negotiates the TLS connection. The default value is false.

Response Timeout

Specifies the timeout for getting a response from the server (the timeout for opening the connection to the server is set by ConnectTimeout). The default value is 30.

If no response is received, the adapter retries repeatedly, up to the number of times given by Failure Timeout divided by Retry Interval.

Retries To Failover

Number of times to retry before trying to failover to the backup HTTP server. A value of -1 means never failover. A value of 0 means to failover for the first retry, 1 means to give it one retry with the normal address before failing over for the second retry, and so on. Default value is 5.

SSL Check Server Identity

Specifies that when making a TLS connection, the adapter should check that the server identity in the certificate matches the name of the system being connecting to. This defaults to specifying that the check should be made. Clear this for test and development systems where the name specified in the TLS certificate does not match the DNS name.

SSL Configuration

The name of an existing TLS configuration to use to authenticate this connection.

When using with the HTTP inbound adapter, choose a server TLS configuration.

When using with the HTTP outbound adapter, choose a client TLS configuration, because the adapter initiates the communication.

To create and manage TLS configurations, use the Management Portal. See InterSystems TLS Guide. The first field on the **Edit SSL/TLS Configuration** page is **Configuration Name**. Use this string as the value for the **SSL Configuration** setting.

SendSuperSession

Controls whether to include the **SuperSessionID** property in the outgoing message. For details, see [Supersessions](#).

URL

URL path to request from the server (not including `http://` or the server address).

Also see [Specifying the Destination Server and URL Path](#).

Use Cookies

Specifies whether to save cookies received in HTTP responses while this adapter is instantiated, and insert them in each subsequent HTTP request.

WriteTimeout

Specifies a timeout value for writes to the web server. If this setting is null, there is no timeout.

